5 . compare the performance of three popular CNN architectures: VGG, ResNet, and GoogLeNet for classification using Dogs vs. Cats dataset.

```
import os
import zipfile
import time
import gc
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import VGG16, ResNet50, Inceptio

dataset_url = "https://storage.googleapis.com/tensorflow-1-public/
zip_path = "cats_and_dogs_filtered.zip"

if not os.path.exists("cats_and_dogs_filtered"):
    print("Downloading dataset...")
    os.system(f'wget -q "{dataset_url}" -O "{zip_path}"')

    print("Extracting dataset...")

    with zipfile.ZipFile(zip_path, "r") as zip_ref:
        zip_ref.extractall(".")

    print("Dataset extracted successfully")

train_dir = "cats_and_dogs_filtered/train"
test_dir = "cats_and_dogs_filtered/validation"

print("Training directory:", train_dir)
print("Validation directory:", test_dir)

IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 5

train_datagen = ImageDataGenerator(
    rescale=1.0 / 255,
    validation_split=0.2
)

test_datagen = ImageDataGenerator(
```

```
    rescale=1.0 / 255
)

train_data = train_datagen.flow_from_directory(
    train_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="training",
    shuffle=True
)

val_data = train_datagen.flow_from_directory(
    train_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="validation",
    shuffle=False
)

test_data = test_datagen.flow_from_directory(
    test_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="binary",
    shuffle=False
)

print("Class indices:", train_data.class_indices)

def build_vgg():
    base = VGG16(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3)
    )

    base.trainable = False

    model = models.Sequential([
        base,
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation="relu"),
        layers.Dropout(0.5),
        layers.Dense(1, activation="sigmoid")
    ])

    return model
```

```
def build_resnet():
    base = ResNet50(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3)
    )

    base.trainable = False

    model = models.Sequential([
        base,
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation="relu"),
        layers.Dropout(0.5),
        layers.Dense(1, activation="sigmoid")
    ])

    return model

def build_googlenet():
    base = InceptionV3(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3)
    )

    base.trainable = False

    model = models.Sequential([
        base,
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation="relu"),
        layers.Dropout(0.5),
        layers.Dense(1, activation="sigmoid")
    ])

    return model

model_builders = {
    "VGG16": build_vgg,
    "ResNet50": build_resnet,
    "GoogLeNet": build_googlenet
}

results = {}

for name, build_model in model_builders.items():
```

```
    print("\n" + "=" * 60)
    print(name)
    print("=" * 60)

    tf.keras.backend.clear_session()
    gc.collect()

    model = build_model()

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    start_time = time.time()

    history = model.fit(
        train_data,
        validation_data=val_data,
        epochs=EPOCHS,
        verbose=1
    )

    training_time = time.time() - start_time

    test_loss, test_accuracy = model.evaluate(
        test_data,
        verbose=0
    )

    results[name] = {
        "accuracy": test_accuracy,
        "loss": test_loss,
        "time": training_time,
        "history": history.history
    }

    print("\nTest Loss:", round(test_loss, 4))
    print("Test Accuracy:", round(test_accuracy * 100, 2), "%")
    print("Training Time:", round(training_time, 2), "seconds")

    del model
    gc.collect()

print("\n" + "=" * 60)
print("FINAL PERFORMANCE COMPARISON")
print("=" * 60)
```

```
for name, result in results.items():
    print(
        f"{name}: "
        f"Accuracy = {result['accuracy'] * 100:.2f}% | "
        f"Loss = {result['loss']:.4f} | "
        f"Time = {result['time']:.2f} seconds"
    )

names = list(results.keys())

accuracies = [
    results[name]["accuracy"] * 100
    for name in names
]

losses = [
    results[name]["loss"]
    for name in names
]

times = [
    results[name]["time"]
    for name in names
]

plt.figure(figsize=(8, 5))

plt.bar(names, accuracies)

plt.xlabel("CNN Architecture")
plt.ylabel("Test Accuracy (%)")
plt.title("VGG16 vs ResNet50 vs GoogLeNet")

plt.ylim(0, 100)

for i, value in enumerate(accuracies):
    plt.text(
        i,
        value + 1,
        f"{value:.2f}%",
        ha="center"
    )

plt.show()

plt.figure(figsize=(8, 5))

plt.bar(names, losses)
```

```
plt.xlabel("CNN Architecture")
plt.ylabel("Loss")
plt.title("Loss Comparison")

plt.show()

plt.figure(figsize=(8, 5))

plt.bar(names, times)

plt.xlabel("CNN Architecture")
plt.ylabel("Training Time (seconds)")
plt.title("Training Time Comparison")

plt.show()

plt.figure(figsize=(10, 6))

for name, result in results.items():
    plt.plot(
        range(1, EPOCHS + 1),
        result["history"]["val_accuracy"],
        marker="o",
        label=name
    )

plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Validation Accuracy Comparison")
plt.legend()
plt.grid()

plt.show()

plt.figure(figsize=(10, 6))

for name, result in results.items():
    plt.plot(
        range(1, EPOCHS + 1),
        result["history"]["val_loss"],
        marker="o",
        label=name
    )

plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("Validation Loss Comparison")
plt.legend()
```

```
plt.legend()
plt.grid()

plt.show()

best_model = max(
    results,
    key=lambda x: results[x]["accuracy"]
)

print("\nBest Performing Model:", best_model)

print(
    "Best Accuracy:",
    round(results[best_model]["accuracy"] * 100, 2),
    "%"
)

print("\nClass Mapping:")
print(train_data.class_indices)
```

```
Downloading dataset...
Extracting dataset...
Dataset extracted successfully
Training directory: cats_and_dogs_filtered/train
Validation directory: cats_and_dogs_filtered/validation
Found 1600 images belonging to 2 classes.
Found 400 images belonging to 2 classes.
Found 1000 images belonging to 2 classes.
Class indices: {'cats': 0, 'dogs': 1}

============================================================
VGG16
============================================================
Downloading data from https://storage.googleapis.com/tensorflow/ke
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Epoch 1/5
50/50 ━━━━━━━━━━━━━━━━━━━━ 1107s 22s/step - accuracy: 0.6481 - los
Epoch 2/5
50/50 ━━━━━━━━━━━━━━━━━━━━ 1106s 22s/step - accuracy: 0.8138 - los
Epoch 3/5
50/50 ━━━━━━━━━━━━━━━━━━━━ 1097s 22s/step - accuracy: 0.8544 - los
Epoch 4/5
50/50 ━━━━━━━━━━━━━━━━━━━━ 1098s 22s/step - accuracy: 0.8712 - los
Epoch 5/5
50/50 ━━━━━━━━━━━━━━━━━━━━ 0s 18s/step - accuracy: 0.8651 - loss:
```